

Modelos, Métricas y Validación

Clase 2 · Regresión, Clasificación y Evaluación rigurosa

Agenda de hoy

01

Métricas primero

Antes de modelos:
¿cómo sabemos si
algo funciona?

02

Regresión

Lineal · Polinómica ·
Regularización L1 L2

03

Clasificación

Logística · Árbol ·
KNN · Multiclase

04

Métricas avanzadas

ROC · AUC · Matriz de
confusión

05

Validación cruzada

Cross-validation y
selección de modelos

Métricas primero

Sin criterio de evaluación, los modelos no tienen sentido

¿Qué hace bueno a un modelo?

- Un modelo que funciona bien en entrenamiento no garantiza nada
- Lo que importa es la GENERALIZACIÓN: predecir bien en datos nuevos
- Para medir eso necesitamos métricas calculadas sobre el conjunto de PRUEBA
- Como estadísticos: piensan en el ECM, sesgo y varianza — ML usa las mismas ideas con otro nombre
 - $MSE \leftrightarrow$ varianza del error de predicción
 - Sesgo del modelo $\leftrightarrow$ underfitting sistemático
- Hay métricas distintas para regresión y clasificación

Métricas para Regresión

MSE

$$(1/m) \sum (\hat{y}_i - y_i)^2$$

✓ Penaliza errores grandes

✗ Unidades al cuadrado — difícil interpretar

RMSE

$$\sqrt{\text{MSE}}$$

✓ Misma unidad que y — interpretable

✗ Sensible a outliers

MAE

$$(1/m) \sum |\hat{y}_i - y_i|$$

✓ Robusto a outliers

✗ No diferenciable en 0

MAPE

$$(100/m) \sum |\hat{y}_i - y_i| / |y_i| \%$$

✓ Porcentaje — comparable entre escalas

✗ Explota si $y_i \approx 0$; asimétrico

R²

$$1 - \frac{\text{SS}_{\text{res}}}{\text{SS}_{\text{to}}}$$

✓ Varianza explicada (0–1)

✗ Siempre sube al añadir variables

MAPE — Interpretación y valores de referencia

$$\text{MAPE} = (100 / m) \cdot \sum |\hat{y}_i - y_i| / |y_i| \rightarrow \text{resultado en \%}$$

MAPE	Interpretación	Ejemplo de contexto
< 5 %	Excelente — predicciones muy precisas	Demanda en manufactura de alta precisión
5 – 10 %	Muy bueno — aceptable en la mayoría de industrias	Forecasting de ventas retail, energía
10 – 20 %	Aceptable — margen razonable de error	Precios de vivienda, demanda agrícola
20 – 50 %	Pobre — el modelo necesita mejorar	Predicción de precios volátiles
> 50 %	Inaceptable — modelo sin valor predictivo	Revisar datos y formulación del problema

⚠ Evitar MAPE cuando y_i pueda ser cero o muy cercano a cero (ej. demanda intermitente). Alternativa: sMAPE o MASE.

Métricas para Clasificación — La matriz de confusión

	Pred. Positivo	Pred. Negativo
Real Positivo	TP ✓	FN — Error Tipo II
Real Negativo	FP — Error Tipo I	TN ✓

Tipo I = FP = Falsa alarma = alpha

Tipo II = FN = Error omision = beta

Accuracy $(TP+TN) / Total$

Fraccion total bien clasificada

Precision $TP / (TP+FP)$

Minimiza Error Tipo I (FP)

Recall $TP / (TP+FN)$

Minimiza Error Tipo II (FN)

F1-Score $2 * P * R / (P+R)$

Balance Tipo I vs Tipo II

Error Tipo I y Error Tipo II — el costo de equivocarse

Error Tipo I (Falso Positivo — FP)

Definición:

Rechazar H_0 siendo verdadera.

Predecir positivo cuando es negativo.

También llamado:

Falsa alarma · α (nivel de significancia)

Ejemplo clínico:

Paciente sano clasificado como enfermo.

Error Tipo II (Falso Negativo — FN)

Definición:

No rechazar H_0 siendo falsa.

Predecir negativo cuando es positivo.

También llamado:

Error de omisión · β · $(1 - \beta = \text{Potencia})$

Ejemplo clínico:

Paciente enfermo clasificado como sano.

Relacion con metricas: Recall (Sensibilidad) = $1 - \beta$ -> minimiza Error Tipo II Especificidad = $1 - \alpha$ -> minimiza Error Tipo I

Trade-off: bajar el umbral reduce Tipo II pero aumenta Tipo I. El contexto define cual priorizar.

Modelos de Regresión

Lineal · Polinómica · Regularización

Regresión Lineal — El modelo base de todo

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n = Xw$$

- w_0 = intercepto (bias); $w_1 \dots w_n$ = coeficientes (pendientes)
- Aprendizaje: minimizar $MSE = (1/m) \sum (\hat{y}_i - y_i)^2$
- Solución analítica (estadística clásica): $\hat{w} = (X^T X)^{-1} X^T y$
- Como estadísticos conocen esto como MCO — scikit-learn lo implementa por defecto
- Supuestos: linealidad, independencia, homocedasticidad, normalidad de residuos

Regresión Lineal en Python

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# 1. Instanciar y entrenar
model = LinearRegression()
model.fit(X_train, y_train)

# 2. Predecir y evaluar
y_pred = model.predict(X_test)
print(f'RMSE: {mean_squared_error(y_test, y_pred, squared=False):.2f}')
print(f'R²: {r2_score(y_test, y_pred):.3f}')

# 3. Inspeccionar coeficientes
print('Intercepto:', model.intercept_)
print('Coeficientes:', dict(zip(feature_names, model.coef_)))
```



Siempre interpretar los coeficientes.

¿Cuándo falla la regresión lineal y qué hacer?

- Problema 1 — Relación no lineal → Regresión Polinómica
 - Agregar features $x^2, x^3 \dots$ Xw sigue siendo lineal en los parámetros → MCO aplica
 - `from sklearn.preprocessing import PolynomialFeatures`
- Problema 2 — Muchas variables o multicolinealidad → Regularización
 - Lasso (L1): añade $\lambda \cdot \sum |w_j|$ a la pérdida → fuerza coeficientes a cero (selección de variables)
 - Ridge (L2): añade $\lambda \cdot \sum w_j^2$ → encoge coeficientes, nunca los anula completamente
 - ElasticNet: combinación L1+L2 → equilibrio entre ambos
- λ es un hiperparámetro → se elige por validación cruzada

Regularización — Lasso, Ridge, ElasticNet

```
from sklearn.linear_model import Ridge, Lasso, ElasticNet
from sklearn.model_selection import cross_val_score

# Ridge (L2) — bueno cuando todas las variables importan un poco
ridge = Ridge(alpha=1.0)    # alpha = lambda (hiperparámetro)
ridge.fit(X_train, y_train)

# Lasso (L1) — bueno para selección de variables (sparse models)
lasso = Lasso(alpha=0.1)
lasso.fit(X_train, y_train)

# Comparar con cross-validation
for name, m in [('Ridge', ridge), ('Lasso', lasso)]:
    cv = cross_val_score(m, X_train, y_train, cv=5, scoring='r2')
    print(f'{name} R2 medio: {cv.mean():.3f} ± {cv.std():.3f}')
```



Un R^2 que baja mucho con Ridge/Lasso respecto a OLS indica posible overfitting en OLS

Modelos de Clasificación

Logística · Árbol de Decisión · KNN

Regresión Logística — clasificación con base estadística sólida

$$P(y=1 | x) = \sigma(Xw) = 1 / (1 + e^{(-Xw)})$$

$\sigma(z)$ toma cualquier valor real
y lo mapea a (0,1)
→ interpretable como probabilidad

- Estima $P(y=1 | x)$ directamente — interpretación probabilística nativa
- Parámetros: coeficientes w (log-odds ratios) — familiares para estadísticos
- Función de pérdida: log-loss = $-\sum [y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)]$
- Umbral de clasificación por defecto = 0.5, pero es ajustable según el costo del error
- Excelente punto de partida: interpretable, probabilístico, robusto

Árbol de Decisión — intuitivo pero cuidadoso

Ventajas

- Divide el espacio de features con preguntas binarias sucesivas
- Criterio de división: Gini o entropía (información)
- No requiere escalar los datos
- Puede manejar variables categóricas y continuas
- Muy interpretable: se puede visualizar y explicar
- Se puede usar para seleccionar variables relevantes (`feature_importances_`)

Limitaciones

- ⚠ Es inestable: pequeños cambios en datos → árbol muy distinto
- ⚠ Propenso a overfitting sin control
- Hiperparámetros clave: `max_depth`, `min_samples_split`
- Solución al overfitting: Random Forest (ensemble de árboles)
- `max_depth=3` suele ser buen punto de partida

K-Vecinos más Cercanos (KNN)

- Idea: para clasificar un punto nuevo, buscar los k puntos más cercanos del training set
- La clase mayoritaria entre esos k vecinos es la predicción
- Hiperparámetro clave: k (número de vecinos)
 - k pequeño ($k=1$): overfitting, muy sensible al ruido
 - k grande: underfitting, fronteras muy suaves
- Requiere escalar features — la distancia euclidiana es sensible a la escala
 - Un feature en millones de pesos dominará sobre uno en metros si no se normaliza
- No tiene fase de entrenamiento explícita — toda la memoria está en los datos (lazy learner)
- Costoso en predicción para datasets grandes

Clasificación — patrón unificado en scikit-learn

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report,
ConfusionMatrixDisplay

# Todos siguen la misma API: fit → predict → evaluate
modelos = {
    'Logística': LogisticRegression(max_iter=1000),
    'Árbol': DecisionTreeClassifier(max_depth=5, random_state=42),
    'KNN': KNeighborsClassifier(n_neighbors=7),
}
for nombre, m in modelos.items():
    m.fit(X_train, y_train)
    print(f'\n{nombre}:')
    print(classification_report(y_test, m.predict(X_test)))
```



classification_report imprime precisión, recall y F1 por clase — úsalo siempre

Curva ROC y AUC

Evaluación de clasificadores más allá del accuracy

Curva ROC — evaluación independiente del umbral

- Eje X: Tasa de Falsos Positivos (FPR) = $FP/(FP+TN)$
- Eje Y: Sensibilidad / Recall = $TP/(TP+FN)$
- Cada punto de la curva = un umbral de clasificación diferente
- Diagonal = clasificador aleatorio (AUC = 0.5)
- Esquina superior izquierda = clasificador perfecto (AUC = 1.0)

AUC = 1.0 Perfecto

AUC > 0.9 Excelente

AUC 0.7–0.9 Bueno

AUC 0.5–0.7 Mediocre

AUC ≤ 0.5 Peor que azar

Ventaja clave: AUC no depende del umbral → compara modelos de forma justa

Validación Cruzada

Estimación robusta del error de generalización

K-Fold Cross-Validation — por qué una sola partición no basta

VAL 1	TRAIN	TRAIN	TRAIN	TRAIN	Fold 1
TRAIN	VAL 2	TRAIN	TRAIN	TRAIN	Fold 2
TRAIN	TRAIN	VAL 3	TRAIN	TRAIN	Fold 3
TRAIN	TRAIN	TRAIN	VAL 4	TRAIN	Fold 4
TRAIN	TRAIN	TRAIN	TRAIN	VAL 5	Fold 5

Resultado: k estimaciones del error → media ± desviación estándar

Si la desviación es alta: el modelo es inestable (señal de alarma)

Validación cruzada y búsqueda de hiperparámetros

```
from sklearn.model_selection import cross_val_score, GridSearchCV

# Cross-validation básica
cv_scores = cross_val_score(LogisticRegression(), X, y, cv=5, scoring='f1')
print(f'F1: {cv_scores.mean():.3f} ± {cv_scores.std():.3f}')

# Búsqueda de hiperparámetros con CV integrada
param_grid = {'max_depth': [3, 5, 7, 10], 'min_samples_split': [2, 5, 10]}
grid = GridSearchCV(DecisionTreeClassifier(), param_grid, cv=5, scoring='f1')
grid.fit(X_train, y_train)

print('Mejores params:', grid.best_params_)
print('Mejor F1 en CV:', grid.best_score_.round(3))
# Evaluar el modelo final SOLO aquí, en test
print('F1 en test:', f1_score(y_test, grid.predict(X_test)).round(3))
```



Si F1 en CV >> F1 en test → el proceso de selección tiene data leakage

¿Qué modelo usar? — guía práctica de selección

Situación	Modelo de arranque	¿Por qué?
Regresión, relación lineal	Regresión lineal / Ridge	Interpretable, MCO exacto
Regresión, no lineal	Regresión polinómica + Lasso	Control de complejidad
Clasificación, necesito probabilidades	Regresión logística	Salida probabilística
Clasificación, quiero interpretabilidad	Árbol de decisión (depth ≤ 5)	Visualizable, explicable
Dataset pequeño, sin supuestos	KNN	Sin parametrización
Siempre validar con...	Cross-validation (k=5 o 10)	Estimación robusta

Resumen de la Clase 2

- Las métricas se calculan SIEMPRE sobre datos de prueba, no de entrenamiento
- Regresión lineal es MCO — un modelo conocido con una interfaz nueva
- Regularización (L1/L2) controla la complejidad: λ se elige por validación cruzada
- Clasificación logística da probabilidades — aprovéchenlas, son estadísticos
- ROC-AUC compara modelos independientemente del umbral de decisión
- Validación cruzada da media $\pm$ desviación del error — siempre reportar ambos

Ejecutar el Notebook de Clase 2 y comparar los modelos sobre el mismo dataset 